

DexbotRL: Agent-Supervised Real-World Reinforcement Learning for Humanoid Dexterous Manipulation

Tianlun HE

Abstract

Vision-language-action models (VLAs) and world action models (WAMs) are rapidly advancing general-purpose robot action generation and the modeling of environmental changes and action consequences, respectively. Real-world reinforcement learning (RL), meanwhile, allows policies to continue adapting through physical interaction. Yet these advances do not guarantee reliable execution in multi-stage, contact-rich tasks. Such tasks require not only action generation, but also continuous tracking of the current task stage, verification that actions produce actual progress, identification of when failures begin, and use of execution outcomes for subsequent policy learning. These challenges motivate a broader research question: how should an agent participate in robot control? This proposal considers two possible modes of integration. An agent may orchestrate existing policies at the task level, or it may itself generate executable action representations such as action chunks and end-effector targets. The two modes may serve different tasks and control timescales, but both require the agent to understand physical execution. DexbotRL begins from this shared prerequisite.

A practical near-term test of this direction is the training-time division of labor between an agent and a policy across different timescales. The policy executes robot actions at the control timescale, while real-world RL updates the policy through physical interaction. After each episode, the agent analyzes the complete trajectory and converts task stage, execution progress, and failure onset into auxiliary supervision for the Critic.

To test this training-time division of labor, DexbotRL develops a real-world RL framework for a Unitree G1 equipped with two Revo2 dexterous hands. The system uses a frozen GR00T model as a multi-task base policy, constructs the RL state with an RL Token, and trains a task-specific Actor-Critic to produce a full 76-D action vector. After each episode, an asynchronous multimodal Agent reads video, proprioception, reference actions, executed actions, and available tactile evidence to generate stage, progress, and failure-onset supervision with temporal localization, references to supporting evidence, and confidence. A deterministic program then compiles these outputs into auxiliary targets for the Critic; the environment reward and TD target remain unchanged. The central hypothesis is that the Agent’s hindsight understanding of complete trajectories can be distilled into the Critic’s pre-execution predictions of action consequences, improving learning efficiency and task reliability under a fixed real-world interaction budget.

1. Research Vision: The Future of Agents and Robot Control

In multi-stage, contact-rich real-world tasks, an agent may participate in robot control through two distinct modes of integration. In the first, an agent orchestrates existing policies; in the second, the agent generates robot actions directly. The distinction is not whether a low-level controller remains in the system, but where action decisions are produced. The first places the agent above policies and lets it decide which capabilities to invoke and how to organize the task. The second makes the agent itself responsible for producing action chunks, end-effector targets, or other executable action representations. The two modes need not replace one another; they may suit different tasks and

coexist at different timescales within the same system.

In the first mode, the agent maintains task state from goals and execution feedback, selects, invokes, or switches policies, and decides whether to continue, retry, or replan when execution stalls or fails. Policies and low-level controllers perform the high-frequency, contact-rich physical execution. The agent outputs subgoals, policy calls, and control-flow decisions rather than robot joint actions. *Claude Plays Robotics* provides direct evidence for this division of labor: general models are markedly more effective when pretrained policies provide a high-level control interface, whereas direct low-level control of high-DoF robots remains difficult. The study estimates that real-time low-level Go2 control requires roughly 83 Hz, while the tested non-reasoning model calls run at only about 0.2–0.4 Hz, leaving a gap of roughly two orders of magnitude [1]. Agent-mediated policy orchestration is therefore the more practical system architecture today.

In the second mode, the agent does not merely select an existing policy. It uses visual observations, robot state, and execution feedback to produce the robot’s next action representation directly. Here, “direct control” does not mean bypassing servos, whole-body controllers, or safety constraints; it means that the action target itself is generated by the agent. VIA demonstrates an early version of this approach. Without robot-specific fine-tuning, a general visual agent uses a 3D interface composed of an RGB-D scene and camera views to generate 6-D end-effector waypoints in a closed loop, while a PI controller executes the trajectory. With Fable 5 and minimal task prompts, VIA achieves 96.7% success on three LIBERO-Goal tasks and 100% on a long-horizon rainbow-arrangement task involving seven blocks [2]. These results remain limited to quasi-static simulated tasks, but they show that a suitable interface can convert general perception, reasoning, and error correction into robot control targets.

The second mode is also supported by progress in inference infrastructure. DeepSeek’s FlashMLA optimizes multi-head latent attention computation during model inference. Its updated kernel reports a 5–15% improvement on compute-bound workloads on H800 GPUs, reaching up to 660 TFLOPS [3]. These measurements describe a specific inference kernel rather than end-to-end agent response time, and they cannot be equated with a robot control rate. They nevertheless show that the computational cost underlying large-model inference still has substantial room for optimization, providing an engineering basis for agents to participate at shorter decision timescales. Extending direct agent-generated actions from quasi-static manipulation to dynamic, contact-rich real-world tasks still requires end-to-end validation of inference latency, action precision, feedback frequency, and safety boundaries.

DexbotRL begins with a prerequisite shared by both modes: understanding physical execution. After each episode, the Agent analyzes the complete trajectory and converts stage, progress, and failure information into auxiliary supervision for the Critic, testing whether an agent’s understanding of execution can improve policy learning. If this mechanism succeeds, the same task-state and execution-evidence interface can support two subsequent research directions: runtime policy selection, retries, and skill organization on one hand, and direct action generation with execution-conditioned correction on the other. DexbotRL therefore first asks whether execution understanding can be fed back into the learning system, and then uses experimental evidence to determine how an agent should participate in robot control.

2. Research Background and Perspective

Robot learning is advancing simultaneously in foundation models, system integration, reinforcement learning, and data generation. These directions are not independent. Foundation models determine what a system can generate; the runtime system determines how model outputs enter physical execution; RL uses interaction to adapt a policy; and data determines which tasks and embodiments the resulting capabilities can cover. For long-horizon, contact-rich humanoid manipulation, a weakness in any one of these layers can limit real-world performance.

2.1 VLA and WAM: Stronger Models Have Not Yet Produced Reliable Complex-Task Execution

VLAs and WAMs expand robot foundation-model capabilities along different paths. A VLA learns a mapping from visual, linguistic, and robot state inputs to actions [4, 5], whereas a WAM learns from video and action sequences how an environment changes over time and in response to actions [6, 7]. Although their modeling objectives differ, both seek to broaden the tasks, embodiments, and motion patterns a model can support. The decisive question is not merely whether a model can generate a more plausible action or future, but whether that capability can be converted consistently into successful execution on complex tasks.

Recent work demonstrates clear gains at the model level. GR00T N1.7 is trained on bimanual, semi-humanoid, and humanoid robot data and uses more than 20,000 hours of first-person human video, allowing the same model to be post-trained for different embodiments [8]. At the implementation level, an embodiment tag selects the corresponding state, action, and normalization configuration; the official documentation also provides a fine-tuning and deployment workflow for Unitree G1 with the SONIC whole-body controller [9]. MotionWAM further uses dynamics features from a video world model and a unified motion latent to represent whole-body actions. Across nine Unitree G1 real-world tasks with matched demonstrations, action spaces, and SONIC interfaces, MotionWAM reports an average success rate of 76.1%, compared with 43.9% for GR00T N1.7 [10]. These results show that cross-embodiment pretraining, video dynamics priors, and whole-body action representations are improving humanoid task performance.

Higher average performance, however, does not establish reliable complex-task execution. HumanoidArena evaluates seven simulated humanoid tasks that require foot participation under a unified 40-D action interface. Under the SONIC tracker, its strongest Diffusion Policy reports average success rates of 52.22% for human–object interaction and 65.83% for human–scene interaction; the VLA-style $\pi_{0.5}$ model achieves 41.67% and 58.33%, respectively. Performance also drops substantially when the underlying tracker changes [11]. Although these simulation results are not directly comparable to MotionWAM’s real-world results, they show continued sensitivity to task distributions, action interfaces, and low-level controllers. Long-horizon tasks still require dedicated mechanisms for phase transitions and accumulated errors: Long-VLA explicitly introduces phase-aware modeling, while LiLo-VLA relies on dynamic replanning and failure recovery [12, 13]. VLAs and WAMs therefore provide stronger action and representation foundations, but not yet a general solution for reliable complex-task execution.

2.2 Agent–Policy Interfaces: From Model Capability to Reliable Execution

Whether a foundation model translates into real-world capability depends not only on the model itself, but also on the interface through which an agent participates in control. Anthropic’s comparison across robot embodiments and control modes shows that the same model performs differently when generating low-level actions, writing controllers, or invoking pretrained policies. For high-DoF robots, high-level control through pretrained policies is more effective, while unnecessary agent intervention in otherwise valid VLA actions can reduce performance [1]. The central problem is therefore not to replace the policy with an agent, but to define appropriate mechanisms for delegation, intervention, and feedback.

VIA demonstrates another interface design: a general agent specifies 6-D end-effector waypoints through a visual 3D interface, while a PI controller executes the trajectory. This allows Fable 5 to complete several simulated tabletop tasks zero-shot, but performance remains limited on the T-block task requiring millimeter-level precision; inference speed and cost also restrict the system largely to quasi-static tasks [2]. An agent can therefore participate in action-target generation, but its practical competence remains jointly determined by the control abstraction, task dynamics, and low-level executor.

This proposal uses **Harness** to denote the runtime mechanisms that support this division of labor.

Harness VLA wraps a frozen VLA as a retryable local contact-manipulation primitive, while an agent performs semantic localization, organizes execution stages, adjusts pre-contact poses, and handles failures. It improves success under disturbances without updating the VLA parameters [14]. Although VISTA studies interactive visual games, it preserves frame-level observations, action history, and revisable task notes, and allows an agent to revisit historical evidence [15]. Together, these systems show that a reliable interface must both organize policy calls and retain a queryable execution history from which the agent can decide whether to continue, retry, or terminate.

These results support a practical near-term division of labor: a low-frequency agent maintains task state, organizes policies, and handles failures, while high-frequency policies and controllers execute stable contact-rich actions. In the works discussed above, the agent primarily handles runtime orchestration and execution recovery. Systematically converting an agent’s understanding of a complete trajectory into auxiliary Critic supervision, and thereby influencing policy learning, has not been evaluated in the systems discussed above [1, 2, 14, 15]. Continued improvement for specific objects and under particular contact conditions therefore requires connecting that execution understanding to real-world reinforcement learning.

2.3 Real-World RL: Exploration and Credit Assignment in High-Dimensional Humanoid Control

Real-world reinforcement learning can use physical interaction to adapt policies to variations in objects, friction, and dynamics that are not covered by pretraining or imitation learning. Existing work first established the feasibility of this path on fixed-base manipulators. ConRFT starts from a pretrained VLA and combines offline behavior cloning, Q-learning, an online consistency policy, and human intervention [16]. HIL-SERL uses demonstrations and online human correction to concentrate training data in regions where the policy is likely to fail [17]. These methods show that, with a strong policy prior and limited human correction, physical interaction can be directed toward targeted policy improvement.

These results do not transfer straightforwardly to humanoids with dexterous hands. ROVE is a recent representative effort extending experience-driven VLA post-training to humanoid dexterous manipulation, using whole-body human intervention, mixed-quality trajectories, and Optimistic Value Estimation for tabletop task learning [18]. It provides initial feasibility evidence while also showing current dependence on human correction and data selection. Unlike a fixed manipulator, a humanoid action perturbation can jointly affect balance, bimanual coordination, and hand contact. Within the scope covered by the real-world studies above, systematic comparisons of high-dimensional whole-body actor exploration across tasks and contact regimes remain limited [16–18].

Exploration determines what data the system can collect; feedback determines what the Critic can learn from those data. VLAC learns a process reward and uses it for Critic and policy optimization [19]. RoboReward uses counterfactual relabeling and temporal clipping to construct failure, near-miss, and partial-progress examples, then trains a vision-language reward model for real-world RL [20]. DenseReward learns frame-level task progress from synthetic failure trajectories and applies the resulting dense feedback to real-world policy optimization [21]. These methods extend supervision beyond final success labels by providing task-level or frame-level reward and progress signals. They do not, however, explicitly factor execution supervision into task stage, progress direction, and failure onset. For multi-stage humanoid tasks, such structured targets may help the Critic assess how intermediate actions affect later outcomes.

DexbotRL addresses these limitations through two corresponding design choices. A multi-task base policy supplies transferable action priors, while each task undergoes separate real-world RL so that one online Actor is not required to explore multiple high-dimensional contact regimes simultaneously. In parallel, an Agent extracts information about task stage, local progress, and failure onset from complete trajectories as structured auxiliary supervision for the Critic. These outputs neither replace environmental rewards nor modify the TD target; they are used to improve action ranking

and credit assignment. The first design constrains real-world exploration, while the second augments process supervision. The quality of the priors supplied by the multi-task base policy, in turn, depends on the coverage and executability of humanoid robot data.

2.4 Humanoid Data: Simulation Generation Is Scaling, but Whole-Body Dexterous Data Remain Difficult to Scale

Humanoid simulation data are moving from manually authored scenes and scripted rules toward model-driven generation. SAGE uses an agent to coordinate scene generators with visual and physics Critics, and generates demonstrations through grasp-pose prediction, inverse kinematics, and navigation planning; its action generation, however, primarily targets Franka pick-and-place and mobile manipulation [22]. GRAIL combines 3D assets, video-generation priors, and motion retargeting to produce more than 20,000 humanoid loco-manipulation sequences with sim-to-real validation on Unitree G1, but its hands remain binary open/close signals mapped to predefined joint configurations [23]. These works demonstrate clear scaling trends in scene construction and whole-body motion data.

Dexterous manipulation data are also advancing. HumanoidGen combines atomic operations, spatial annotations, and LLM reasoning and search to generate 26-DoF bimanual arm-and-hand manipulation trajectories, but focuses primarily on tabletop settings; novel asset categories outside its library and previously unannotated atomic manipulation types still require manual annotation [24]. Do As I Do recovers hand-object interactions from monocular human video and retargets them into multi-finger robot actions; its reconstruction and retargeting focus on hand-object interaction rather than complete scenes or humanoid whole-body motion [25]. Taken together, the cited methods cover complementary parts of scene generation, whole-body trajectories, and local hand-object interaction, but none jointly provides continuous multi-finger actions, credible contact dynamics, whole-body coordination, and real-world executability [22–25]. Within these pipelines, contact-rich generation also remains dependent on predefined atomic operations, manual annotations, and retargeting rules, limiting low-cost scaling.

This gap does not diminish the value of simulation data; it clarifies the division of labor between generated data and real-world learning. Generated data can supply task distributions, action priors, and policy initialization, while real-world RL can correct friction, slip, compliance, and control errors through physical interaction. Task-specific real-world RL on top of a multi-task base policy is therefore a key link between scalable simulated data and reliable physical execution, and a direct motivation for DexbotRL.

3. DexbotRL: Research Methodology and Experimental Design

3.1 Research Question and Core Hypothesis

DexbotRL studies single-task adaptation of multi-task robot foundation models in high-DoF real-world manipulation tasks. A pretrained base policy can supply transferable visual-language representations and action priors, but these priors do not directly accommodate a specific object, contact condition, or robot dynamics. Real-world RL can continue refining the policy through physical interaction, yet it faces both costly exploration in a high-dimensional action space and difficult credit assignment under sparse rewards. The project first establishes a multi-task base policy plus single-task 76-D RL as its baseline system, and treats Agent-Critic auxiliary supervision as the central variable to be tested independently.

The core research question is: with the base policy, action interface, and real-world interaction budget held fixed, can information about task stage, local progress, and failure onset extracted by an Agent from complete trajectories improve the Critic’s action ranking and value calibration, and ultimately increase the single-task policy’s success rate and learning efficiency? The experiment

changes only the form of supervision applied to the Critic, separating the effect of Agent supervision from changes in policy architecture, action space, or interaction data.

The core hypothesis is that complete trajectories contain execution information that sparse environmental rewards do not express explicitly. Distilling these time-localized, evidence-grounded hindsight judgments into the Critic may improve its ability to assess downstream outcomes before an action is executed. The hypothesis is supported only if the complete evidence chain holds: higher-quality Agent labels, improved Critic ranking and calibration, and better policy performance under a fixed real-world interaction budget.

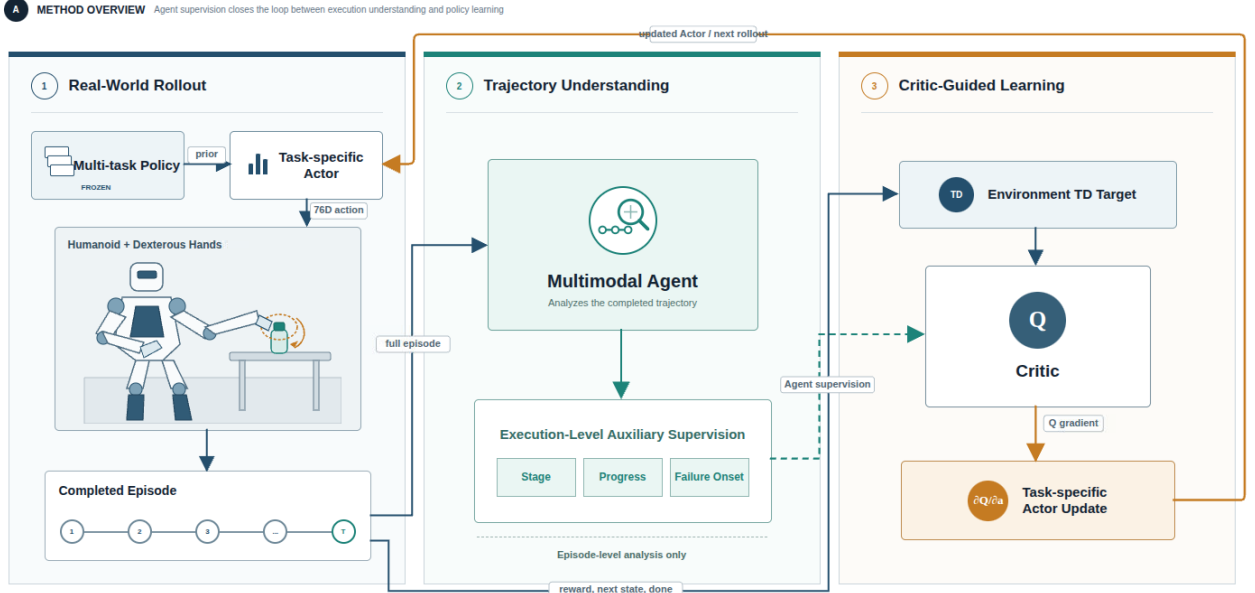


Figure 1: Overview of DexbotRL. Each completed episode yields both environmental TD signals and Agent-generated execution-level auxiliary supervision. The two signals jointly train the Critic, whose Q-value gradient updates the task-specific Actor.

3.2 Policy Baseline: Single-Task Real-World RL on a Multi-Task Base Policy

DexbotRL places cross-task capability and within-task exploration at different levels. A frozen GR00T model serves as the shared multi-task base policy and provides visual-language representations and reference actions. For each task, DexbotRL trains a separate RL Token and Actor–Critic using task-specific real-world interaction data. This design retains general priors from the base policy while avoiding joint online exploration across heterogeneous contact regimes, concentrating a limited real-world interaction budget on the task at hand.

The RL Token converts the GR00T representation into a compact state $z_{rl,t}$ for value learning. The Actor combines this state with robot proprioception p_t and the GR00T reference action a_t^{ref} to generate the full action executed by the robot:

$$\begin{aligned}
 z_{rl,t} &= f_{\psi}\left(h_t^{\text{GR00T}}\right), \\
 a_t &= \pi_{\theta}\left(z_{rl,t}, p_t, a_t^{\text{ref}}\right) = [a_t^{\text{body}}, a_t^{\text{left}}, a_t^{\text{right}}] \in \mathbb{R}^{76}, \\
 a_t^{\text{body}} &\in \mathbb{R}^{64}, \quad a_t^{\text{left}}, a_t^{\text{right}} \in \mathbb{R}^6.
 \end{aligned}$$

The 64-D body latent is decoded by the whole-body controller, while the two 6-D actions control the left and right Revo2 hands. The reference action is an Actor input, not the final command. The Actor directly generates the complete 76-D action rather than restricting adaptation to a hand-only

or other local residual on the base-policy action, preserving joint adaptation across body posture, arm motion, and hand contact.

The Critic estimates long-term return from $z_{rl,t}$, proprioception, and a candidate action, and is trained using a standard temporal-difference (TD) target defined by the environment reward, next state, and termination signal. GR00T, the RL Token, the full-action Actor, and the TD Critic together define the DexbotRL task baseline. Subsequent Agent experiments preserve the state interface, action space, Actor architecture, and environment reward so that performance differences can be attributed to the additional Critic supervision.

3.3 Core Method: Agent-Critic Auxiliary Supervision from Complete Trajectories

Standard temporal-difference learning defines value targets using environment rewards, successor states, and termination signals, but these signals do not fully describe execution in multi-stage tasks. Final success or failure does not directly reveal whether stable contact was established, whether an action produced progress or regression, or when an observable failure first appeared. DexbotRL therefore uses a multimodal Agent as an asynchronous trajectory-analysis teacher. From completed episodes, the Agent extracts execution-level information that is not represented explicitly in standard TD targets, without redefining the task reward or participating in the current rollout.

A deterministic outer workflow orchestrates the Agent. It fixes the order of trajectory analysis, evidence queries, result verification, and label compilation, and constrains tool calls, retries, and abstention. The multimodal model makes judgments only within designated analysis nodes; it cannot add new processing stages, call the environment, or control the robot. The model, prompts, tool interfaces, and output versions are fixed before an experiment so that analysis of an episode can be recorded and replayed. The design does not depend on unconstrained dialogue among multiple agents; one constrained multimodal model performs the different stages of trajectory analysis.

After an episode, a deterministic evidence builder aligns video, proprioception, GR00T reference actions, executed actions, environmental rewards, termination causes, and available tactile records with the actual action chunks, and constructs a common temporal index. The Agent first reads the task contract and a trajectory overview, then uses registered read-only interfaces to inspect relevant video intervals, state changes, and differences between reference and executed actions. Access to a complete trajectory therefore means that the Agent can query evidence from anywhere in the episode; it does not require placing all raw video and numerical sequences into a single model context.

The task contract specifies the available task stages, stage transitions, success conditions, and observable failure events. It is established on an independent trajectory set, reviewed by a human, and version-locked before formal annotation. When analyzing a new episode, the Agent must use the fixed stage and event vocabulary rather than redefine the task for each trajectory. The analysis first locates observable state changes and candidate failure intervals, after which a separate verification pass checks whether video, proprioceptive, and action evidence agree. When a judgment lacks sufficient support, the Agent must narrow the interval, revise the judgment, or abstain.

For each action chunk, the Agent addresses three questions relevant to value learning: the task stage before execution, whether the action moves the task forward, leaves it stalled, or moves it backward, and whether an observable failure begins within the action interval. Each judgment retains its time range, multimodal evidence, and confidence information. Natural-language explanations are kept for audit and error analysis but are not provided to the Critic. The Agent may abstain independently on each of the three fields; for example, the stage may be clear even when occlusion prevents a reliable judgment about slip.

The Agent produces time-localized semantic judgments rather than rewards or training tensors. A deterministic label compiler uses the episode identifier and action-chunk start to align those judgments exactly with state-action samples in replay. Stage judgments become stage-classification targets; forward, stalled, and backward judgments become progress targets; and failure onsets be-

come failure-onset targets. Only evidence-verified judgments enter training. Abstentions, evidence conflicts, and incompletely aligned intervals are excluded from the corresponding auxiliary objective with independent masks. Failure to detect an event is not automatically a negative label; a negative label for failure onset is generated only when the complete action interval has been validly examined.

The three auxiliary prediction heads share the Critic’s state–action representation with the Q function. The Critic objective is

$$\mathcal{L}_{\text{critic}} = \mathcal{L}_{\text{TD}} + \lambda_s \mathcal{L}_{\text{stage}} + \lambda_p \mathcal{L}_{\text{progress}} + \lambda_f \mathcal{L}_{\text{failure}}.$$

Here, $\mathcal{L}_{\text{TD}}$ is still computed only from environment rewards, successor states, and termination signals. The auxiliary objectives train stage recognition, action-progress assessment, and failure-onset prediction. Each objective is evaluated only when its corresponding label is valid and is weighted by label reliability after verification and calibration. The Agent does not choose loss weights or alter the interpretation of Q as the expected long-term return under the environment reward.

Complete trajectories and post-action outcomes are available only to the offline teacher. The online Critic can access only the state and candidate action available before execution. The Actor does not read Agent labels directly and continues to learn through the Critic’s Q values. The Agent’s influence is therefore restricted to a clear path: complete trajectories yield hindsight execution supervision, the Critic learns to assess candidate-action consequences in advance, and the Actor improves through Q-value gradients. The method ultimately tests not whether an Agent can explain a completed trajectory, but whether such trajectory understanding improves the Critic’s credit assignment and, in turn, policy performance under a fixed real-world interaction budget.

3.4 Experimental Design and Staged Validation

3.4.1 Task Settings and Primary Endpoints

The initial study uses two tasks—opening a bottle and operating an air-conditioner remote control—to cover complementary contact regimes. Bottle opening requires the robot to stabilize the bottle and continuously rotate the cap, testing bimanual coordination and sustained control under changing friction and slip. The remote-control task requires the robot to grasp and reorient the device, localize a target button, and verify the trigger outcome, testing small-target localization and precise, discrete contact. Both tasks use the same GR00T base policy, with a separately trained single-task Actor–Critic for each task.

Bottle opening is the primary experiment for the proposed method; the remote-control task tests whether the conclusion transfers to a different contact structure. The primary endpoint is success rate after a fixed real-world interaction budget. Bottle opening succeeds when the cap reaches a predefined open state while the bottle remains stable. The remote-control task succeeds when the specified button is actuated and produces the intended observable state change. Before the experiment, each condition fixes the number of interaction steps, episodes, evaluation trials, independent training repetitions, and reset rules. Success rates are reported with confidence intervals. Learning curves, interaction and reset costs, failure types, and safety stops are secondary metrics.

3.4.2 Baseline System Qualification

The Agent-Critic experiment is built on a qualified 76-D single-task RL baseline. Four configurations are compared:

Setting	State and policy interface	Question
GR00T-Direct	Execute the GR00T reference action directly	Can the multi-task base policy complete the task without RL adaptation?
RL-Raw	Train a 76-D Actor–Critic directly on GR00T features	Does single-task real-world RL provide adaptation gains?
RL-Pool	Use a parameter-free summary matched to the RL Token width	Are gains explained only by compression or input dimensionality?
RL-Token	Train the 76-D Actor–Critic with a learned RL Token	Does the RL Token provide attributable representation gains?

This stage establishes a reliable task baseline rather than the main Agent-Critic claim. The 76-D Actor–Critic must improve consistently over GR00T-Direct under the same budget before Agent-Critic experiments begin. The independent effect of the RL Token is reported from its matched comparisons with RL-Raw and RL-Pool. After qualification, the base-policy checkpoint, state interface, Actor architecture and initialization protocol, action space, and environment reward are held fixed across subsequent comparisons.

3.4.3 Core Agent-Critic Comparison

The core experiment varies the form of Critic supervision while holding the training-data volume, real-world interaction budget, and all other training settings constant:

Setting	Critic supervision	Purpose
TD-only	Standard TD target defined by the environment reward	Measure the qualified task baseline
One-shot	The same multimodal model reads the trajectory once and generates labels	Control for gains from the multimodal model itself
Agent-Critic	Full workflow with evidence queries, consistency checks, confidence, and abstention	Isolate the contribution of the evidence-grounded Agent workflow

All three configurations use the same Critic backbone, initialization protocol, replay data, update budget, and online interaction budget. TD-only disables the auxiliary objectives; One-shot and Agent-Critic use the same auxiliary heads and losses, differing only in how their labels are generated.

All three configurations are compared first on bottle opening. If Agent-Critic improves supervision quality, Critic metrics, and fixed-budget task success, the TD-only and Agent-Critic conditions are then compared on the remote-control task to test whether the result replicates under a different contact regime. Removing the stage, progress, and failure objectives one at a time, and separating tactile input as Agent evidence from tactile input to the online Critic, are secondary ablations. They are screened first in simulation and offline experiments rather than being combined with the core real-world robot experiment.

3.4.4 Evidence Chain and Stage Gates

Evaluation follows the causal chain from supervision to Critic to policy. On a human-reviewed subset, the supervision layer measures the accuracy of stage, progress, and failure labels, evidence validity, effective coverage, and abstention quality. The Critic layer measures action ranking, value calibration, and failure-onset prediction. The policy layer measures task success and learning efficiency under the fixed interaction budget. A final performance change is attributed to Agent-Critic supervision only when an improvement at one layer propagates to the next.

Validation proceeds through four gates. First, simulation checks that the 76-D action interface, RL Token, and auxiliary objectives train stably. Second, an offline pilot uses fixed successful, failed,

and timed-out trajectories; the Agent workflow must improve label quality over one-shot annotation and improve the Critic’s action ranking or calibration. Third, the fixed-budget real-world robot experiment is run on bottle opening. Fourth, the main comparison is replicated on the remote-control task. All stages use consistent task definitions, trajectory formats, and evaluation criteria. Simulation and offline results screen mechanisms; the final claim is determined by real-world robot performance.

4. Project Feasibility

DexbotRL consists of modules that can be validated independently. A frozen GR00T model provides multi-task representations and action priors, so downstream training can focus on a task-specific RL Token and Actor–Critic rather than relearning full visual-language capabilities from limited real-world data. Agent supervision is generated asynchronously after each episode, adding no inference latency to the online control loop and leaving the optimization target defined by the environment reward unchanged. Representation learning, policy adaptation, and Critic supervision can therefore be evaluated with matched controls, making their contributions attributable.

The experimental setup combines simulation, a teleoperation and data-collection pipeline, a Unitree G1 equipped with two Revo2 dexterous hands, and GPU compute. Simulation supports rapid screening of state representations, action interfaces, and auxiliary objectives. Teleoperation and data infrastructure organize successful, failed, and timed-out trajectories while preserving video, proprioception, and executed actions as evidence. The robot platform tests learning under real contact, dynamics mismatch, and whole-body coordination. A shared task definition and trajectory format allow offline supervision to enter subsequent Critic training and real-world evaluation directly.

The primary uncertainties are high-dimensional real-world exploration, Agent label error, and simulation-to-reality transfer. Multi-task action priors, single-task RL, and a fixed interaction budget constrain the exploration space, while safety stops and failure logs control real-robot cost. References to supporting evidence, confidence, and abstention filter Agent outputs, which remain auxiliary to the Critic. Simulation findings are verified independently on the physical system rather than being treated as substitutes for real execution. Each risk thus has a corresponding control, metric, and stage gate, enabling DexbotRL to progress from offline mechanism validation to closed-loop real-world learning.

5. Expected Contributions

The project is expected to make three contributions:

1. **Method.** Investigate evidence-grounded, time-localized execution supervision generated by an Agent, and test whether it improves real-world Actor–Critic learning on top of a VLA base policy.
2. **System.** Establish reproducible interfaces among GR00T, the RL Token, the Critic, and the Agent, and evaluate task performance, sample efficiency, and safety boundaries on humanoid dexterous manipulation.
3. **Research insight.** Clarify the division of responsibilities in embodied systems: which information should enter a policy, which should supervise a critic, and when an agent should orchestrate policies or directly generate control outputs.

In the longer term, world models, policies, and agents may jointly form a closed loop for real-world robots. A world model predicts possible futures; an agent interprets goals and execution state, then either selects and organizes policies or directly generates action targets at an appropriate timescale; policies and low-level controllers provide stable physical execution where needed. DexbotRL studies a component shared by both modes: how an understanding of execution can be fed back into the

learning system.

References

- [1] Berman et al., “Claude Plays Robotics,” Anthropic Research, 2026. [Online]. Available: <https://www.anthropic.com/research/claude-plays-robotics>. Accessed: Aug. 16, 2026.
- [2] Hu et al., “VIA: Visual Interface Agent for Robot Control,” *arXiv preprint arXiv:2607.11119*, 2026.
- [3] DeepSeek-AI, “FlashMLA: Efficient Multi-head Latent Attention Kernels,” GitHub repository, 2025. [Online]. Available: <https://github.com/deepseek-ai/FlashMLA>. Accessed: Aug. 16, 2026.
- [4] Bjorck et al., “GR00T N1: An Open Foundation Model for Generalist Humanoid Robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [5] Kim et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” in *Proc. 8th Conf. Robot Learning (CoRL)*, PMLR, vol. 270, pp. 2679–2713, 2025.
- [6] Ye et al., “World Action Models are Zero-shot Policies,” *arXiv preprint arXiv:2602.15922*, 2026.
- [7] Gao et al., “DreamDojo: A Generalist Robot World Model from Large-Scale Human Videos,” in *Proc. 43rd Int. Conf. Machine Learning (ICML)*, PMLR, vol. 306, 2026. [Online]. Available: <https://arxiv.org/abs/2602.06949>. Accessed: Aug. 16, 2026.
- [8] NVIDIA, “NVIDIA Isaac GR00T N1.7: Open Reasoning VLA Model for Humanoid Robots,” Hugging Face Blog, 2026. [Online]. Available: <https://huggingface.co/blog/nvidia/gr00t-n1-7>. Accessed: Aug. 16, 2026.
- [9] NVIDIA GEAR Team, “VLA Workflow: Collect, Fine-tune, Deploy,” *GR00T-WholeBodyControl* documentation, 2026. [Online]. Available: https://nvlabs.github.io/GR00T-WholeBodyControl/tutorials/vla_workflow.html. Accessed: Aug. 16, 2026.
- [10] Zheng et al., “MotionWAM: Towards Foundation World Action Models for Real-Time Humanoid Loco-Manipulation,” *arXiv preprint arXiv:2606.09215*, 2026.
- [11] Wang et al., “HumanoidArena: Benchmarking Egocentric Hierarchical Whole-body Learning,” *arXiv preprint arXiv:2606.17833*, 2026.
- [12] Fan et al., “Long-VLA: Unleashing Long-Horizon Capability of Vision Language Action Model for Robot Manipulation,” *arXiv preprint arXiv:2508.19958*, 2025.
- [13] Yang et al., “LiLo-VLA: Compositional Long-Horizon Manipulation via Linked Object-Centric Policies,” *arXiv preprint arXiv:2602.21531*, 2026.
- [14] Zhang et al., “Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents,” *arXiv preprint arXiv:2607.08448*, 2026.
- [15] Han et al., “VISTA: A Visual Harness for Reasoning in an Interactive World,” Project website, 2026. [Online]. Available: <https://vista-research.github.io/>. Accessed: Aug. 16, 2026.
- [16] Chen et al., “ConRFT: A Reinforced Fine-tuning Method for VLA Models via Consistency Policy,” *arXiv preprint arXiv:2502.05450*, 2025.
- [17] Luo et al., “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” *arXiv preprint arXiv:2410.21845*, 2024.
- [18] Xiao et al., “ROVE: Unlocking Human Interventions for Humanoid Manipulation via Reinforcement Learning,” *arXiv preprint arXiv:2606.17011*, 2026.
- [19] Zhai et al., “A Vision-Language-Action-Critic Model for Robotic Real-World Reinforcement Learning,” *arXiv preprint arXiv:2509.15937*, 2025.
- [20] Lee et al., “RoboReward: General-Purpose Vision-Language Reward Models for Robotics,” *arXiv preprint arXiv:2601.00675*, 2026.
- [21] Fang et al., “DenseReward: Dense Reward Learning via Failure Synthesis for Robotic Manipulation,” *arXiv preprint arXiv:2607.13033*, 2026.
- [22] Xia et al., “SAGE: Scalable Agentic 3D Scene Generation for Embodied AI,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 22358–22368, 2026.
- [23] Xie et al., “GRAIL: Generating Humanoid Loco-Manipulation from 3D Assets and Video Priors,” *arXiv preprint arXiv:2606.05160*, 2026.
- [24] Jing et al., “HumanoidGen: Data Generation for Bimanual Dexterous Manipulation via LLM Reasoning,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [25] Paliwal et al., “Do As I Do: Dexterous Manipulation Data from Everyday Human Videos,” *arXiv preprint arXiv:2606.19333*, 2026.